USB PD CTS ENGINEERING CHANGE NOTICE

NOTICE: Any Company or Companies submitting a USB Power Delivery ECR proposal must be one of the following: a Promoter or Contributor of the USB 3.0 and 2.0 Specifications who have completed the USB Power Delivery addendum. If a group of Companies is submitting an ECR proposal, each company must be either a Promoter or Contributor of the USB 3.0 and 2.0 Specifications who have completed the USB Power Delivery addendum.

SPECIFICATION REVISIONS AND ADDENDA: At any point in time, there shall only be one current version of the USB PD CTS, termed the production version. At the same time, there may also be proposed revisions to the specification's design which are not yet approved and shall be held confidential as deemed necessary by the USB 3.0 and USB 2.0 Promoters and within the Group of Working Committee(s).

PROCEDURES FOR SUBMITTING PROPOSALS: Both members of the USB Implementers Forum as a whole and members of the USB 3.0 and USB 2.0 Promoters may submit requests to revise the USB PD CTS Specification. Such a request may be rejected or may result in a USB PD Engineering Change Notice (ECN), which is the official way USB specifications may be changed.

FORMAT OF PROPOSAL: The originator of a request to alter the USB PD CTS Specification may do so by posting this to the USB Power Delivery Compliance working group for review. Once the proposal has been reviewed by the working group it will be passed to the USB 3.0 and 2.0 Promoters for approval to publish.

RESUBMISSION AND APPEAL: The originator of a request that was not approved can redraft the original request. Rewritten proposal will be treated as a new proposal and will be evaluated using the procedures described above. The originator of a request that was not approved can also submit an appeal to the USB 3.0 and 2.0 Promoters. The appeal must be made in writing and addressed to the Secretary of the USB Implementers Forum.

ABOUT THE ENGINEERING CHANGE REQUEST FORM:
The Purpose of this Engineering Change Request Form is to expedite the review process of the proposal by providing explanations, background information, and examples of the proposed changes at a high level. This form serves as an executive summary to the actual proposal.

STEPS ON HOW TO SUBMIT A USB PD ENGINEERING CHANGE REQUEST:

1) Please fill out the Engineering Change Request Form on the following pages completely:
   a) Detail the names and contact details for each of the ECR contributors
   b) Update the ECR Title
   c) Give a minimum of 2-3 sentences for each description on the form outlining the background to the ECR
2) For each section/table/figure to be updated:
   a) Detail the section number, starting page and figure/table number to be updated as appropriate.
   b) Detail existing text under "From Text"
   c) Detail changed text under "To Text"
3) Save the file as "USB PD CTS 1.0 R 1" followed by the ECR Title as per step 1)b)
4) Post the ECR in the USB PD CTS Documents section under "ECR | New ECRs".
   a) This ECR will then be reviewed by the Power Delivery Compliance Working Group.
   b) Revisions to the ECR originating from the review should be submitted as document revision of the original ECR using "Add new document".

USB PD CTS ENGINEERING CHANGE NOTICE

Name: Yoon Lee Email: yoonlee@apple.com

Company: Apple Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

Name: _____ Email: _____

Company: _____ Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

Name: _____ Email: _____

Company: _____ Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

Name: _____ Email: _____

Company: _____ Mailstop: _____

Address: _____

City: _____ State/Province: _____

Country: _____ Zip/Postal Code: _____

Phone: _____ FAX: _____

USB PD CTS ENGINEERING CHANGE NOTICE

Title: TEST.PD.PROT.ALL3.6
Applied to: USB PD CTS Q3, 2026

Brief description of the functional changes proposed:
Update TEST.PD.PROT.ALL3.6 to allow Soft Reset to be issued by UUT upon tChunkSenderResponse timeout.

Benefits as a result of the proposed changes:
Check UUT behavior per base spec definition on protocol error triggered by tChunkSenderResponse timeout.

An assessment of the impact to the existing revision and systems that currently conform to the USB specification:
None

An analysis of the hardware implications:
None

An analysis of the software implications:
None

An analysis of the compliance testing implications:
Tester to update the test per the ECR. COMMON.CHECK.PD.5 shall not apply to the Soft_Reset Message received from UUT upon tChunkSenderResponse max.

An analysis of the Vendor Info File (VIF) implications:
None

Actual Change Requested

TEST.PD.PROT.ALL3.6 ChunkSenderResponseTimer Timeout

From Text:

TEST.PD.PROT.ALL3.6 ChunkSenderResponseTimer

3. The tester checks the response.
    a. If the UUT is a Cable Plug, the check fails if the UUT does not respond with either ***Ignore*** or a request for a Chunk of Data Block message. [TEST.PD.PROT.ALL3.6#1]
    b. If the UUT is a Source or Sink, the Tester checks response: [TEST.PD.PROT.ALL3.6#2]
        i. The check fails if Sink or Source UUT does not respond with either *Not_Supported* or a request for a Chunk of Data Block message.
        ii. The check fails if a *Not_Supported* Message received, and it is not received between *tChunkingNotSupported* min and *tChunkingNotSupported* max.
        iii. The check fails if a request for a Chunk of a Data Block is received, and it is not received within *tChunkReceiverRequest* max.
    c. The test stops here if the UUT does not respond with a request for chunk message. Upon receipt of a request response from the UUT, the Tester sends the requested chunk to the UUT. This step is repeated until the Tester has sent 4 chunk messages and it intentionally does not send the 5th chunk message to the UUT.
    d. The Tester runs the following additional checks the request chunk message: [TEST.PD.PROT.ALL3.6#3]
        i. For Message Header
            1. Number of Data Objects = 1
            2. Message Type = 11111b
        ii. For Extended Message Header
            1. Bits 14…11 Chunk Number = next chunk in the series
            2. Bit 10 Request Chunk = 1
            3. Bits 8…0 Data Size = 0
4. The Tester waits for *tChunkSenderResponse* max + **Error! Reference source not found.** before re-sending to the UUT the first chunk message of the same chunked extended message defined in step 2.
5. The tester checks the response.
    a. If the UUT is a Cable Plug, the check fails if the UUT does not respond with either ***Ignore*** or a request for a Chunk of Data Block message. [TEST.PD.PROT.ALL3.6#4]
    b. If the UUT is a Source or Sink, the Tester checks the response [TEST.PD.PROT.ALL3.6#5]
        i. The check fails if Sink or Source UUT does not respond with either *Not_Supported* or a request for a Chunk of Data Block message.
        ii. The check fails if a *Not_Supported* Message received, it is not received between *tChunkingNotSupported* min and *tChunkingNotSupported* max.
        iii. The check fails if a request for a Chunk of a Data Block is received, it is not received within *tChunkReceiverRequest* max.
    c. Upon receipt of the response from the UUT, the Tester sends the requested chunk to the UUT.
    d. The Tester runs the following additional checks on the request chunk message: [TEST.PD.PROT.ALL3.6#6]
        i. For Message Header
            1. Number of Data Objects = 1
            2. Message Type = 11111b
        ii. For Extended Message Header
            1. Bits 14…11 Chunk Number = next chunk in the series
            2. Bit 10 Request Chunk = 1
            3. Bits 8…0 Data Size = 0

To Text:

TEST.PD.PROT.ALL3.6 ChunkSenderResponseTimer

3. The tester checks the response.
    a. If the UUT is a Cable Plug, the check fails if the UUT does not respond with either ***Ignore*** or a request for a Chunk of Data Block message. [TEST.PD.PROT.ALL3.6#1]
    b. If the UUT is a Source or Sink, the Tester checks response: [TEST.PD.PROT.ALL3.6#2]
        i. The check fails if Sink or Source UUT does not respond with either *Not_Supported* or a request for a Chunk of Data Block message.
        ii. The check fails if a *Not_Supported* Message received, and it is not received between *tChunkingNotSupported* min and *tChunkingNotSupported* max.
        iii. The check fails if a request for a Chunk of a Data Block is received, and it is not received within *tChunkReceiverRequest* max.
    c. The test stops here if the UUT does not respond with a request for chunk message. Upon receipt of a request response from the UUT, the Tester sends the requested chunk to the UUT. This step is repeated until the Tester has sent 4 chunk messages and it intentionally does not send the 5th chunk message to the UUT.
    d. The Tester runs the following additional checks the request chunk message: [TEST.PD.PROT.ALL3.6#3]
        i. For Message Header
            1. Number of Data Objects = 1
            2. Message Type = 11111b
        ii. For Extended Message Header
            1. Bits 14…11 Chunk Number = next chunk in the series
            2. Bit 10 Request Chunk = 1
            3. Bits 8…0 Data Size = 0
4. If the UUT is a Source or Sink:
    a. The Tester waits for *tChunkSenderResponse* max + *tProtErrSoftReset* max + **Error! Reference source not found.**, measured from the last bit of the EOP of the *GoodCRC* Message corresponding to the UUT's Chunk Request for the 5th chunk, and observes the UUT's response.
        i. If a *Soft_Reset* Message is received,
            1. the Tester replies with a *GoodCRC* Message and then an Accept within *tReceiverResponse* max.
            2. the Tester re-establishes the explicit contract. If the UUT is a Source, the check fails if the UUT does not send a *Source_Capabilities* Message within *tFirstSourceCap*; if the UUT is a Sink, the Tester sends a *Source_Capabilities* Message and completes the negotiation using COMMON.PROC.PD.2. [TEST.PD.PROT.ALL3.6#7]
            After the Explicit contract has been re-established, proceeds to step 6.
        ii. The check fails if a *Soft_Reset* Message is not received and the test stops here. [TEST.PD.PROT.ALL3.6#8]
5. If the UUT is a Cable Plug:
    a. The Tester waits *tChunkSenderResponse* max + **Error! Reference source not found.** measured from the last bit of the EOP of the *GoodCRC* Message corresponding to the UUT's Chunk Request for the 5th chunk, and observes the UUT's response.
    b. The check fails if the UUT sends a *Soft_Reset* Message and the test stops here. [TEST.PD.PROT.ALL3.6#9]
6. The tester resends to the UUT the first chunk message of the same chunked extended message defined in step 2.
7. The tester checks the response.
    a. If the UUT is a Cable Plug, the check fails if the UUT does not respond with either ***Ignore*** or a request for a Chunk of Data Block message. [TEST.PD.PROT.ALL3.6#4]
    b. If the UUT is a Source or Sink, the Tester checks the response [TEST.PD.PROT.ALL3.6#5]

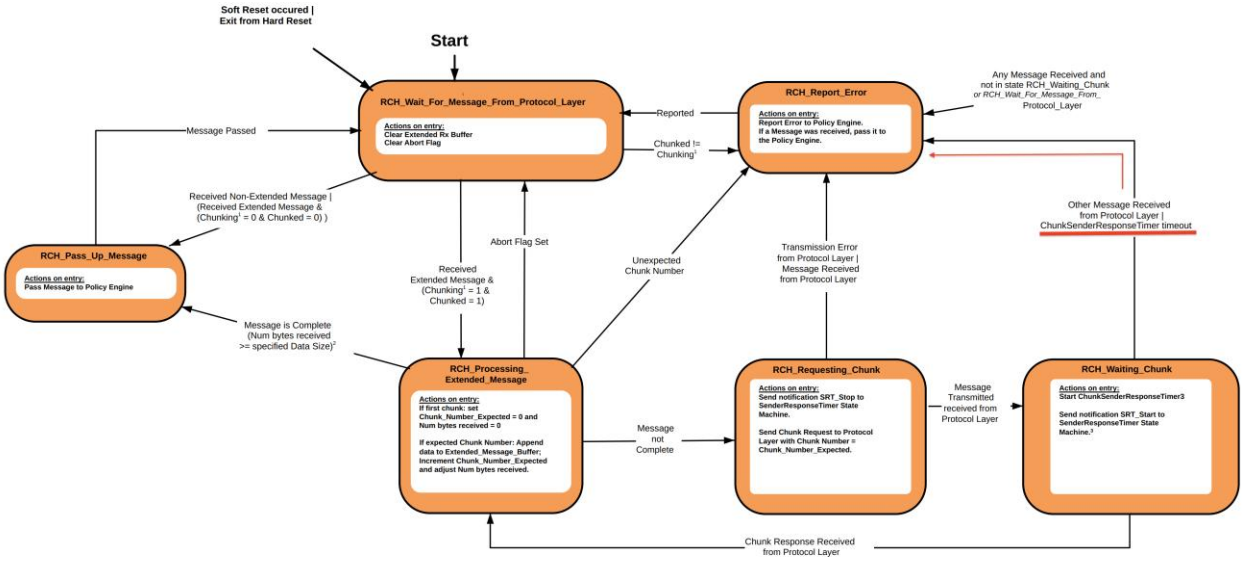
i. The check fails if Sink or Source UUT does not respond with either *Not_Supported* or a request for a Chunk of Data Block message.
ii. The check fails if a *Not_Supported* Message received, it is not received between *tChunkingNotSupported* min and *tChunkingNotSupported* max.
iii. The check fails if a request for a Chunk of a Data Block is received, it is not received within *tChunkReceiverRequest* max.

c. Upon receipt of the response from the UUT, the Tester sends the requested chunk to the UUT.
d. The Tester runs the following additional checks on the request chunk message: [TEST.PD.PROT.ALL3.6#6]
    i. For Message Header
        1. Number of Data Objects = 1
        2. Message Type = 11111b
    ii. For Extended Message Header
        1. Bits 14…11 Chunk Number = next chunk in the series
        2. Bit 10 Request Chunk = 1
        3. Bits 8…0 Data Size = 0

USB PD CTS ENGINEERING CHANGE NOTICE

Spec References:

Figure 9.4. Chunked Rx State Diagram



9.2.5.2.1. PE_SNK_Send_Soft_Reset State

The PE_SNK_Send_Soft_Reset State **Shall** be entered from any State when:

- A Protocol Error on SOP is detected by the Protocol Layer during an AMS (see Section 7.1.1) or

7.1.1. Soft Reset and Protocol Error

A Soft_Reset Message is used to cause a Soft Reset of the communication protocol when it has broken down in some way. The breakdown in communication or the Protocol Error **May** take several forms:

- An Unexpected Message is received during an AMS.
- An expected Message fails to arrive.
- A timeout occurs while waiting for a Message or event.

The Soft Reset **Shall Not** have any impact on power supply operation but is used to correct a Protocol Error occurring during an Atomic Message Sequence (AMS). After a Soft Reset has completed, an Explicit Contract Negotiation occurs, in order to re-establish PD Communication and to bring State operation for both Port Partners back to either the PE_SNK_Ready or PE_SRC_Ready states as appropriate (see Section 9.2.5). The Soft Reset **May** be triggered by either Port Partner in response to the Protocol Error. An Accept Message is sent by the recipient of a Soft Reset Message to indicate that it has completed the Soft Reset.

7.31.8.2. tProtErrSoftReset

If the Protocol Error occurs that causes the Source or Sink to send a Soft_Reset Message, the transmission of the Soft_Reset Message **Shall** be completed within tProtErrSoftReset of the EOP of the GoodCRC Message sent in response to the Message that caused the Protocol Error.

USB PD CTS ENGINEERING CHANGE NOTICE